Valentin LE GALL
Elia LEPREVOST

Bases de données réparties

Nano-Satellites

PHASE 3
PL/SQL & Package pkg_nanoOrbit
Projet

ALTN83-2526BSP01 - Bases de données réparties
Christophe CROISANT

1) Script Paliers 1–5 **3**
Palier 1 : 3
Palier 2 : 5
Palier 3 : 6
Palier 4 : 9
Palier 5 : 12
2) Script SPEC : pkg_nanoOrbit **18**
3) Script BODY : pkg_nanoOrbit **21**
4) Script de validation **30**

1) Script Paliers 1–5

Palier 1 :

Exercice 1 — Affichage d'un message de bienvenue et comptage des entités

Nous avons créé un bloc anonyme PL/SQL contenant trois variables numériques destinées à stocker :

- le nombre de satellites ;
- le nombre de stations au sol ;
- le nombre de missions.

Ces valeurs ont été obtenues à l'aide de trois requêtes SELECT COUNT(*) INTO ..., puis affichées avec DBMS_OUTPUT.PUT_LINE.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_nb_satellites NUMBER;
    v_nb_stations   NUMBER;
    v_nb_missions   NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_nb_satellites FROM SATELLITE;
    SELECT COUNT(*) INTO v_nb_stations   FROM STATION_SOL;
    SELECT COUNT(*) INTO v_nb_missions   FROM MISSION;

    DBMS_OUTPUT.PUT_LINE('*** Bienvenue dans NanoOrbit ***');
    DBMS_OUTPUT.PUT_LINE('Nombre de satellites : ' || v_nb_satellites);
    DBMS_OUTPUT.PUT_LINE('Nombre de stations   : ' || v_nb_stations);
    DBMS_OUTPUT.PUT_LINE('Nombre de missions   : ' || v_nb_missions);
END;
/
```

Résultat obtenu

```
*** Bienvenue dans NanoOrbit ***
Nombre de satellites : 5
Nombre de stations   : 3
Nombre de missions   : 3
```

Analyse

Le bloc s'exécute correctement et permet de vérifier rapidement que les données de référence ont bien été chargées dans la base.

Exercice 2 — Récupération des caractéristiques du satellite SAT-001 avec SELECT INTO

Dans ce second exercice, nous avons utilisé SELECT INTO afin de récupérer les informations d'un satellite précis, ici SAT-001. Les attributs sélectionnés ont été stockés dans des variables PL/SQL typées à partir de la table SATELLITE grâce à %TYPE. Les données ont ensuite été affichées avec DBMS_OUTPUT.PUT_LINE.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_nom       SATELLITE.nom_satellite%TYPE;
    v_statut    SATELLITE.statut%TYPE;
    v_batterie  SATELLITE.capacite_batterie%TYPE;
    v_orbite    SATELLITE.id_orbite%TYPE;
BEGIN
    SELECT nom_satellite, statut, capacite_batterie, id_orbite
    INTO v_nom, v_statut, v_batterie, v_orbite
    FROM SATELLITE
    WHERE id_satellite = 'SAT-001';

    DBMS_OUTPUT.PUT_LINE('Satellite : ' || v_nom);
    DBMS_OUTPUT.PUT_LINE('Statut    : ' || v_statut);
    DBMS_OUTPUT.PUT_LINE('Batterie  : ' || v_batterie || ' Wh');
    DBMS_OUTPUT.PUT_LINE('Orbite    : ' || v_orbite);
END;
/
```

Résultat obtenu

```
Satellite : NanoOrbit-Alpha
Statut    : Opérationnel
Batterie  : 20 Wh
Orbite        : ORB-001
```

Analyse

Cet exercice montre l'utilisation correcte de SELECT INTO dans un bloc anonyme lorsque la requête ne retourne qu'une seule ligne. Le satellite SAT-001 correspond, dans le jeu de données de référence, à NanoOrbit-Alpha, de statut Opérationnel, avec une capacité batterie de 20 Wh et une orbite ORB-001.

Palier 2 :

Exercice 3 — Lecture d'une ligne complète de SATELLITE avec %ROWTYPE

Dans cet exercice, nous avons déclaré une variable PL/SQL de type SATELLITE%ROWTYPE. Ce type permet de représenter une ligne complète de la table SATELLITE dans une seule variable. Nous avons ensuite utilisé un SELECT * INTO pour charger les informations du satellite SAT-001, puis affiché son statut et sa capacité batterie avec DBMS_OUTPUT.PUT_LINE. L'énoncé demande précisément de lire une ligne complète de SATELLITE et d'afficher ces deux informations.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_sat SATELLITE%ROWTYPE;
BEGIN
    SELECT *
    INTO v_sat
    FROM SATELLITE
    WHERE id_satellite = 'SAT-001';

    DBMS_OUTPUT.PUT_LINE('Satellite : ' || v_sat.nom_satellite);
    DBMS_OUTPUT.PUT_LINE('Statut    : ' || v_sat.statut);
    DBMS_OUTPUT.PUT_LINE('Batterie  : ' || v_sat.capacite_batterie || '
Wh');
END;
/
```

Résultat obtenu

```
Satellite : NanoOrbit-Alpha
Statut    : Opérationnel
Batterie  : 20 Wh
```

Analyse

L'utilisation de %ROWTYPE simplifie la lecture d'un enregistrement complet, car elle évite de déclarer une variable par colonne. Cette solution est particulièrement adaptée lorsque plusieurs attributs de la table doivent être manipulés dans un même bloc PL/SQL.

Exercice 4 — Gestion des valeurs NULL avec NVL

Dans cet exercice, nous avons affiché la résolution d'un instrument en remplaçant les valeurs NULL par le texte N/A. Pour cela, nous avons utilisé la fonction Oracle NVL. Le cas

visé par le sujet est l'instrument INS-AIS-01, dont la résolution est volontairement absente dans le jeu de données, car un récepteur AIS ne produit pas d'image. Le document de référence précise explicitement que ce cas sert à l'exercice du palier 2 consacré à NVL.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_resolution_affichee VARCHAR2(20);
    v_modele              INSTRUMENT.modele%TYPE;
BEGIN
    SELECT modele, NVL(TO_CHAR(resolution), 'N/A')
    INTO v_modele, v_resolution_affichee
    FROM INSTRUMENT
    WHERE ref_instrument = 'INS-AIS-01';

    DBMS_OUTPUT.PUT_LINE('Instrument  : ' || v_modele);
    DBMS_OUTPUT.PUT_LINE('Résolution  : ' || v_resolution_affichee);
END;
/
```

Résultat obtenu

```
Instrument  : ShipTrack-V2
Résolution  : N/A
```

Analyse

L'utilisation de NVL permet d'éviter un affichage vide ou ambigu lorsqu'une colonne contient NULL. Dans notre script, la valeur numérique de resolution a été convertie en texte avec TO_CHAR, puis remplacée par N/A lorsque la colonne était vide.

Palier 3 :

Exercice 5 — Catégorisation avec IF / ELSIF

Dans cet exercice, nous avons récupéré le statut du satellite SAT-005 puis utilisé une structure conditionnelle IF / ELSIF / ELSE afin d'afficher un message correspondant à son état opérationnel.

Code

```
SET SERVEROUTPUT ON;

DECLARE
```

```
    v_statut SATELLITE.statut%TYPE;
BEGIN
    SELECT statut
    INTO v_statut
    FROM SATELLITE
    WHERE id_satellite = 'SAT-005';

    IF v_statut = 'Opérationnel' THEN
        DBMS_OUTPUT.PUT_LINE('Satellite pleinement actif.');
    ELSIF v_statut = 'En veille' THEN
        DBMS_OUTPUT.PUT_LINE('Satellite temporairement inactif.');
    ELSIF v_statut = 'Désorbité' THEN
        DBMS_OUTPUT.PUT_LINE('Satellite hors service définitif.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Statut non reconnu.');
    END IF;
END;
/
```

Résultat obtenu

```
Satellite hors service définitif.
```

Analyse

La structure conditionnelle permet d'associer un traitement différent à chaque statut satellite.

Exercice 6 — Traitement multi-cas avec CASE

Nous avons utilisé l'instruction CASE afin de traiter plusieurs valeurs possibles du type d'orbite ORB-001. Cette structure est plus lisible qu'une succession de IF lorsque les cas sont fixes et connus à l'avance.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_type_orbite ORBITE.type_orbite%TYPE;
BEGIN
    SELECT type_orbite
    INTO v_type_orbite
    FROM ORBITE
    WHERE id_orbite = 'ORB-001';

    CASE v_type_orbite
```

```
        WHEN 'SSO' THEN
            DBMS_OUTPUT.PUT_LINE('Orbite adaptée à l''observation
polaire.');
        WHEN 'LEO' THEN
            DBMS_OUTPUT.PUT_LINE('Orbite basse terrestre.');
        WHEN 'GEO' THEN
            DBMS_OUTPUT.PUT_LINE('Orbite géostationnaire.');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Type orbital non prévu.');
    END CASE;
END;
/
```

Résultat obtenu

```
Orbite adaptée à l'observation polaire.
```

Analyse

Le CASE améliore la lisibilité du script et facilite la maintenance du code.

Exercice 7 — Boucle FOR de simulation

Dans cet exercice, nous avons utilisé une boucle FOR pour simuler plusieurs durées de passage satellite et calculer le volume de données transféré pour chacune d'elles.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_debit NUMBER := 2.5; -- Mo/s
    v_volume NUMBER;
BEGIN
    FOR i IN 1..3 LOOP
        v_volume := (i * 5) * 60 * v_debit;

        DBMS_OUTPUT.PUT_LINE(
            'Durée : ' || (i * 5) || ' min -> Volume : ' || v_volume ||
' Mo'
        );
    END LOOP;
END;
/
```

Résultat obtenu

```
Durée : 5 min -> Volume : 750 Mo
Durée : 10 min -> Volume : 1500 Mo
Durée : 15 min -> Volume : 2250 Mo
```

Analyse

La boucle FOR permet d'automatiser les calculs répétitifs.

Palier 4 :

Exercice 8 — Utilisation de SQL%ROWCOUNT

Nous avons effectué une mise à jour du statut du satellite SAT-004, puis utilisé SQL%ROWCOUNT afin d'afficher le nombre de lignes réellement modifiées.

Code

```
SET SERVEROUTPUT ON;

BEGIN
    UPDATE SATELLITE
    SET statut = 'Opérationnel'
    WHERE id_satellite = 'SAT-004';

    DBMS_OUTPUT.PUT_LINE(
        SQL%ROWCOUNT || ' ligne(s) mise(s) à jour.'
    );
END;
/
```

Résultat obtenu

```
1 ligne(s) mise(s) à jour.
```

Analyse

L'attribut implicite SQL%ROWCOUNT permet de connaître immédiatement le nombre de lignes impactées par la dernière instruction DML.

Exercice 9 — Parcours avec curseur implicite

Nous avons utilisé une boucle FOR ... IN (SELECT ...) afin de parcourir automatiquement tous les satellites de la base sans déclaration explicite de curseur.

Code

```
SET SERVEROUTPUT ON;

BEGIN
    FOR rec IN (
        SELECT id_satellite, nom_satellite, statut
        FROM SATELLITE
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            rec.id_satellite || ' - ' ||
            rec.nom_satellite || ' - ' ||
            rec.statut
        );
    END LOOP;
END;
/
```

Résultat obtenu

```
SAT-001 - NanoOrbit-Alpha - Opérationnel
SAT-002 - NanoOrbit-Beta - Opérationnel
SAT-003 - NanoOrbit-Gamma - Opérationnel
SAT-004 - NanoOrbit-Delta - En veille
SAT-005 - NanoOrbit-Epsilon - Désorbité
```

Analyse

Le curseur implicite est particulièrement adapté pour les traitements simples de parcours ligne par ligne.

Il simplifie fortement le code tout en restant très lisible.

Exercice 10 — Curseur explicite

Dans cet exercice, nous avons déclaré un curseur explicite c_sat, puis utilisé OPEN, FETCH, EXIT WHEN et CLOSE afin de parcourir les satellites de manière détaillée.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_sat IS
        SELECT id_satellite, nom_satellite, statut
        FROM SATELLITE;
```

```
    v_id      SATELLITE.id_satellite%TYPE;
    v_nom     SATELLITE.nom_satellite%TYPE;
    v_statut  SATELLITE.statut%TYPE;
BEGIN
    OPEN c_sat;

    LOOP
        FETCH c_sat INTO v_id, v_nom, v_statut;
        EXIT WHEN c_sat%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            v_id || ' - ' || v_nom || ' - ' || v_statut
        );
    END LOOP;

    CLOSE c_sat;
END;
/
```

Résultat obtenu

```
SAT-001 - NanoOrbit-Alpha - Opérationnel
SAT-002 - NanoOrbit-Beta - Opérationnel
SAT-003 - NanoOrbit-Gamma - Opérationnel
SAT-004 - NanoOrbit-Delta - En veille
SAT-005 - NanoOrbit-Epsilon - Désorbité
```

Analyse

Le curseur explicite offre un contrôle plus fin sur le parcours du jeu de résultats.

Il est particulièrement utile lorsque plusieurs traitements intermédiaires doivent être appliqués entre deux lectures.

Exercice 11 — Curseur paramétré

Nous avons créé un curseur paramétré recevant le code d'une station en paramètre afin d'afficher toutes les fenêtres de communication associées.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_fenetres(p_station VARCHAR2) IS
        SELECT id_fenetre, volume_donnees, statut
```

```
        FROM FENETRE_COM
        WHERE code_station = p_station;

BEGIN
    FOR rec IN c_fenetres('GS-TLS-01')
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Fenêtre ' || rec.id_fenetre ||
            ' - Volume : ' || NVL(TO_CHAR(rec.volume_donnees), 'N/A') ||
            ' - Statut : ' || rec.statut
        );
    END LOOP;
END;
/
```

Résultat obtenu

```
Fenêtre 2 - Volume : 890 - Statut : Réalisée
Fenêtre 4 - Volume : N/A - Statut : Planifiée
Fenêtre 5 - Volume : N/A - Statut : Planifiée
```

Analyse

Le curseur paramétré améliore la réutilisabilité du code, car il permet de réappliquer le même traitement à plusieurs stations.

Palier 5 :

Exercice 12 — Gestion des exceptions prédéfinies

Dans cet exercice, nous avons réalisé un SELECT INTO sur la table SATELLITE en sécurisant le traitement avec un bloc EXCEPTION. L'objectif était de gérer explicitement le cas où aucun satellite ne correspond à l'identifiant fourni (NO_DATA_FOUND), ainsi que toute autre erreur (OTHERS).

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_nom      SATELLITE.nom_satellite%TYPE;
    v_statut   SATELLITE.statut%TYPE;
BEGIN
    SELECT nom_satellite, statut
    INTO v_nom, v_statut
```

```
    FROM SATELLITE
    WHERE id_satellite = 'SAT-999';

    DBMS_OUTPUT.PUT_LINE('Satellite : ' || v_nom || ' - ' || v_statut);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aucun satellite ne correspond à cet
identifiant.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur Oracle : ' || SQLERRM);
END;
/
```

Résultat obtenu

```
Aucun satellite ne correspond à cet identifiant.
```

Analyse

Ce traitement permet d'éviter l'arrêt brutal du bloc PL/SQL lorsqu'un identifiant inexistant est fourni.

Exercice 13 — Validation métier avec RAISE_APPLICATION_ERROR

Nous avons mis en place un contrôle procédural avant insertion d'une fenêtre de communication. La validation porte sur trois points : le satellite doit être autorisé à communiquer, la station doit être active, et les règles de planification métier doivent être respectées. Le sujet cite explicitement ces contrôles pour cet exercice.

Code

```
SET SERVEROUTPUT ON;

DECLARE
    v_statut_satellite SATELLITE.statut%TYPE;
    v_statut_station   STATION_SOL.statut%TYPE;
BEGIN
    SELECT statut
    INTO v_statut_satellite
    FROM SATELLITE
    WHERE id_satellite = 'SAT-005';

    IF v_statut_satellite <> 'Opérationnel' THEN
        RAISE_APPLICATION_ERROR(
            -20010,
```

```
            'Satellite non autorisé pour une nouvelle fenêtre.'
        );
    END IF;

    SELECT statut
    INTO v_statut_station
    FROM STATION_SOL
    WHERE code_station = 'GS-SGP-01';

    IF v_statut_station <> 'Active' THEN
        RAISE_APPLICATION_ERROR(
            -20011,
            'Station non active pour la planification.'
        );
    END IF;

    DBMS_OUTPUT.PUT_LINE('Validation réussie.');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

Résultat obtenu

```
ORA-20010: Satellite non autorisé pour une nouvelle fenêtre.
```

Analyse

La procédure de validation permet de lever une erreur explicite et compréhensible pour l'utilisateur via RAISE_APPLICATION_ERROR.

Ce test est cohérent avec le référentiel, où SAT-005 est désorbité et GS-SGP-01 est en maintenance.

Exercice 14 — Procédure afficher_statut_satellite

Nous avons créé une procédure standalone afficher_statut_satellite(p_id IN) chargée d'afficher le statut, l'orbite et les instruments embarqués d'un satellite donné. Le sujet impose explicitement cette procédure comme livrable du socle du palier 5.

Code

```
CREATE OR REPLACE PROCEDURE afficher_statut_satellite (
    p_id IN SATELLITE.id_satellite%TYPE
```

```oracle
)
IS
    v_nom      SATELLITE.nom_satellite%TYPE;
    v_statut   SATELLITE.statut%TYPE;
    v_orbite   SATELLITE.id_orbite%TYPE;
BEGIN
    SELECT nom_satellite, statut, id_orbite
    INTO v_nom, v_statut, v_orbite
    FROM SATELLITE
    WHERE id_satellite = p_id;

    DBMS_OUTPUT.PUT_LINE('Satellite : ' || v_nom);
    DBMS_OUTPUT.PUT_LINE('Statut    : ' || v_statut);
    DBMS_OUTPUT.PUT_LINE('Orbite    : ' || v_orbite);
    DBMS_OUTPUT.PUT_LINE('Instruments :');

    FOR rec IN (
        SELECT e.ref_instrument, i.type_instrument,
e.etat_fonctionnement
        FROM EMBARQUEMENT e
        JOIN INSTRUMENT i
          ON i.ref_instrument = e.ref_instrument
        WHERE e.id_satellite = p_id
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            '- ' || rec.ref_instrument || ' / ' ||
            rec.type_instrument || ' / ' ||
            rec.etat_fonctionnement
        );
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Satellite introuvable : ' || p_id);
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;
/
```

Test

```oracle
BEGIN
    afficher_statut_satellite('SAT-001');
END;
```

```
/
```

Résultat obtenu

```
Satellite : NanoOrbit-Alpha
Statut    : Opérationnel
Orbite        : ORB-001
Instruments :
- INS-CAM-01 / Caméra optique / Nominal
- INS-IR-01 / Infrarouge / Nominal
```

Analyse

Cette procédure centralise l'affichage des informations essentielles d'un satellite.

Dans le jeu de référence, SAT-001 est opérationnel sur ORB-001 et embarque notamment INS-CAM-01 et INS-IR-01.

Exercice 15 — Procédure mettre_a_jour_statut

Nous avons créé une procédure mettre_a_jour_statut(p_id IN, p_statut IN, p_ancien_statut OUT) permettant de modifier le statut d'un satellite tout en retournant son ancien statut via un paramètre OUT.

Code

```
CREATE OR REPLACE PROCEDURE mettre_a_jour_statut (
    p_id              IN  SATELLITE.id_satellite%TYPE,
    p_statut          IN  SATELLITE.statut%TYPE,
    p_ancien_statut   OUT SATELLITE.statut%TYPE
)
IS
BEGIN
    SELECT statut
    INTO p_ancien_statut
    FROM SATELLITE
    WHERE id_satellite = p_id;

    UPDATE SATELLITE
    SET statut = p_statut
    WHERE id_satellite = p_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20020, 'Aucune mise à jour
effectuée.');
    END IF;
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20021, 'Satellite introuvable.');
    WHEN OTHERS THEN
        RAISE;
END;
/
```

Test

```
DECLARE
    v_ancien_statut SATELLITE.statut%TYPE;
BEGIN
    mettre_a_jour_statut('SAT-004', 'Opérationnel', v_ancien_statut);
    DBMS_OUTPUT.PUT_LINE('Ancien statut : ' || v_ancien_statut);
END;
/
```

Résultat obtenu

```
Ancien statut : En veille
```

Analyse

Cette procédure illustre l'usage combiné de paramètres IN et OUT. Elle permet de récupérer l'état précédent avant modification, ce qui est utile pour la traçabilité métier.

Dans le jeu initial, SAT-004 est en statut En veille, ce qui en fait un bon cas de test.

Exercice 16 — Fonction calculer_volume_session

Nous avons créé une fonction calculer_volume_session(p_id_fenetre IN) RETURN NUMBER qui retourne le volume théorique d'une session de communication à partir de la durée de la fenêtre et du débit maximal de la station associée. Le sujet demande explicitement ce calcul sous la forme débit × durée.

Code

```
CREATE OR REPLACE FUNCTION calculer_volume_session (
    p_id_fenetre IN FENETRE_COM.id_fenetre%TYPE
)
RETURN NUMBER
IS
    v_duree      FENETRE_COM.duree%TYPE;
    v_debit      STATION_SOL.debit_max%TYPE;
```

```
    v_volume     NUMBER;
BEGIN
    SELECT f.duree, s.debit_max
    INTO v_duree, v_debit
    FROM FENETRE_COM f
    JOIN STATION_SOL s
      ON s.code_station = f.code_station
    WHERE f.id_fenetre = p_id_fenetre;

    v_volume := v_duree * v_debit;

    RETURN v_volume;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20030, 'Fenêtre introuvable.');
END;
/
```

Test

```
BEGIN
    DBMS_OUTPUT.PUT_LINE(
        'Volume théorique : ' || calculer_volume_session(1)
    );
END;
/
```

Résultat obtenu

```
Volume théorique : 168000
```

Analyse

Cette fonction illustre le rôle d'une fonction PL/SQL : retourner une valeur calculée exploitable dans un autre traitement.

Le jeu de données contient plusieurs fenêtres numérotées, ce qui permet de tester directement la fonction sur une donnée réelle.

2) Script SPEC : pkg_nanoOrbit

Le livrable L3-B consiste à créer la spécification publique du package pkg_nanoOrbit. Cette SPEC définit l'interface du package en déclarant les types, constantes, procédures et

fonctions qui seront ensuite implémentés dans le BODY. Le sujet précise que la SPEC doit compiler sans erreur avant toute écriture du corps du package.

Nous avons créé un package Oracle nommé pkg_nanoOrbit contenant :

- un type public t_stats_satellite destiné à retourner des statistiques synthétiques sur un satellite ;
- trois constantes métier ;
- quatre procédures de gestion ;
- trois fonctions de calcul et de synthèse.

Code

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE pkg_nanoOrbit IS

    ----------------------------------------------------------------
    -- Type public : statistiques synthétiques d'un satellite
    ----------------------------------------------------------------
    TYPE t_stats_satellite IS RECORD (
        nb_fenetres         NUMBER,
        volume_total        NUMBER,
        duree_moy_secondes  NUMBER
    );

    ----------------------------------------------------------------
    -- Constantes métier publiques
    ----------------------------------------------------------------
    c_statut_min_fenetre CONSTANT SATELLITE.statut%TYPE :=
'Opérationnel';
    c_duree_max_fenetre  CONSTANT NUMBER := 900;
    c_seuil_revision     CONSTANT NUMBER := 50;

    ----------------------------------------------------------------
    -- Procédures publiques
    ----------------------------------------------------------------
    PROCEDURE planifier_fenetre(
        p_id_satellite    IN  SATELLITE.id_satellite%TYPE,
        p_code_station    IN  STATION_SOL.code_station%TYPE,
        p_datetime_debut  IN  FENETRE_COM.datetime_debut%TYPE,
        p_duree           IN  FENETRE_COM.duree%TYPE,
        p_id_fenetre      OUT FENETRE_COM.id_fenetre%TYPE
    );

    PROCEDURE cloturer_fenetre(
```

```
        p_id_fenetre       IN FENETRE_COM.id_fenetre%TYPE,
        p_volume_donnees   IN FENETRE_COM.volume_donnees%TYPE
    );

    PROCEDURE affecter_satellite_mission(
        p_id_satellite IN PARTICIPATION.id_satellite%TYPE,
        p_id_mission   IN PARTICIPATION.id_mission%TYPE,
        p_role         IN PARTICIPATION.role_satellite%TYPE
    );

    PROCEDURE mettre_en_revision(
        p_id_satellite IN SATELLITE.id_satellite%TYPE
    );

    ----------------------------------------------------------------
    -- Fonctions publiques
    ----------------------------------------------------------------
    FUNCTION calculer_volume_theorique(
        p_id_fenetre IN FENETRE_COM.id_fenetre%TYPE
    ) RETURN NUMBER;

    FUNCTION statut_constellation
    RETURN VARCHAR2;

    FUNCTION stats_satellite(
        p_id_satellite IN SATELLITE.id_satellite%TYPE
    ) RETURN t_stats_satellite;

END pkg_nanoOrbit;
/
SHOW ERRORS;
```

Vérification de compilation

```
Package created.
```

Analyse

La SPEC constitue le contrat public du package. Elle permet de séparer l'interface de l'implémentation, ce qui améliore la lisibilité, la maintenance et la réutilisation du code PL/SQL.

Les constantes déclarées traduisent directement plusieurs règles du sujet, notamment :

- le statut minimal autorisé pour la planification d'une fenêtre ;

- la durée maximale d'une fenêtre fixée à 900 secondes ;
- un seuil métier de révision utilisé pour les traitements avancés.

Le type t_stats_satellite facilitera le retour structuré de statistiques sur un satellite, tandis que les procédures et fonctions exposées couvrent les principales opérations métier du package.

Conclusion

La SPEC du package pkg_nanoOrbit a été définie conformément à l'énoncé. Elle sert de base au livrable L3-C, qui consistera à implémenter le BODY du package, puis au livrable L3-D, dédié au scénario de validation.

3) Script BODY : pkg_nanoOrbit

Le livrable L3-C consiste à implémenter le corps du package pkg_nanoOrbit, c'est-à-dire la logique métier associée aux procédures et fonctions déclarées dans la SPEC. Le sujet précise que les procédures doivent réutiliser les triggers de la phase 2 lorsque cela est pertinent, et que la gestion des exceptions doit s'appuyer sur des codes -20xxx cohérents.

Nous avons implémenté les sept sous-programmes du package :

- quatre procédures de gestion métier ;
- trois fonctions de calcul et de synthèse.

Le BODY repose sur les mécanismes déjà présents dans le schéma NanoOrbit :

- les triggers de validation des fenêtres ;
- le trigger d'historisation des changements de statut ;
- les contraintes DDL définies en phase 2.

Code

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE BODY pkg_nanoOrbit IS

    ----------------------------------------------------------------
    -- planifier_fenetre
    -- Insère une nouvelle fenêtre de communication.
    -- Les triggers de Phase 2 contrôlent :
    --   - satellite désorbité
    --   - station en maintenance
    --   - chevauchements
```

```
    ----------------------------------------------------------------
    PROCEDURE planifier_fenetre(
        p_id_satellite    IN  SATELLITE.id_satellite%TYPE,
        p_code_station    IN  STATION_SOL.code_station%TYPE,
        p_datetime_debut  IN  FENETRE_COM.datetime_debut%TYPE,
        p_duree           IN  FENETRE_COM.duree%TYPE,
        p_id_fenetre      OUT FENETRE_COM.id_fenetre%TYPE
    )
    IS
    BEGIN
        IF p_duree IS NULL OR p_duree <= 0 THEN
            RAISE_APPLICATION_ERROR(
                -20040,
                'Durée invalide : une fenêtre doit avoir une durée
strictement positive.'
            );
        END IF;

        IF p_duree > c_duree_max_fenetre THEN
            RAISE_APPLICATION_ERROR(
                -20041,
                'Durée invalide : la durée maximale autorisée est de '
|| c_duree_max_fenetre || ' secondes.'
            );
        END IF;

        INSERT INTO FENETRE_COM (
            id_fenetre,
            datetime_debut,
            duree,
            elevation_max,
            volume_donnees,
            statut,
            id_satellite,
            code_station
        )
        VALUES (
            NULL,
            p_datetime_debut,
            p_duree,
            0,
            NULL,
            'Planifiée',
            p_id_satellite,
```

```
            p_code_station
        )
        RETURNING id_fenetre INTO p_id_fenetre;

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            RAISE_APPLICATION_ERROR(
                -20042,
                'Insertion refusée : identifiant de fenêtre dupliqué.'
            );
        WHEN OTHERS THEN
            RAISE;
    END planifier_fenetre;


    ----------------------------------------------------------------
    -- cloturer_fenetre
    -- Passe une fenêtre en statut Réalisée et renseigne le volume.
    ----------------------------------------------------------------
    PROCEDURE cloturer_fenetre(
        p_id_fenetre       IN FENETRE_COM.id_fenetre%TYPE,
        p_volume_donnees   IN FENETRE_COM.volume_donnees%TYPE
    )
    IS
        v_statut FENETRE_COM.statut%TYPE;
    BEGIN
        IF p_volume_donnees IS NULL OR p_volume_donnees < 0 THEN
            RAISE_APPLICATION_ERROR(
                -20043,
                'Volume invalide : il doit être renseigné et positif ou
nul.'
            );
        END IF;

        SELECT statut
        INTO v_statut
        FROM FENETRE_COM
        WHERE id_fenetre = p_id_fenetre
        FOR UPDATE;

        IF v_statut = 'Réalisée' THEN
            RAISE_APPLICATION_ERROR(
                -20044,
                'Cette fenêtre est déjà clôturée.'
```

```oracle_plsql
        );
    END IF;

    UPDATE FENETRE_COM
    SET statut = 'Réalisée',
        volume_donnees = p_volume_donnees
    WHERE id_fenetre = p_id_fenetre;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20045,
            'Aucune fenêtre mise à jour.'
        );
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(
            -20046,
            'Fenêtre introuvable.'
        );
    WHEN OTHERS THEN
        RAISE;
END cloturer_fenetre;


-------------------------------------------------------------------
-- affecter_satellite_mission
-- Affecte un satellite à une mission avec un rôle.
-- Le trigger T4 doit bloquer l'affectation à une mission terminée.
-------------------------------------------------------------------
PROCEDURE affecter_satellite_mission(
    p_id_satellite IN PARTICIPATION.id_satellite%TYPE,
    p_id_mission   IN PARTICIPATION.id_mission%TYPE,
    p_role         IN PARTICIPATION.role_satellite%TYPE
)
IS
BEGIN
    IF p_role IS NULL OR TRIM(p_role) IS NULL THEN
        RAISE_APPLICATION_ERROR(
            -20047,
            'Le rôle du satellite dans la mission est obligatoire.'
        );
    END IF;
```

```
        INSERT INTO PARTICIPATION (
            id_satellite,
            id_mission,
            role_satellite,
            commentaire
        )
        VALUES (
            p_id_satellite,
            p_id_mission,
            p_role,
            'Ajout via pkg_nanoOrbit.affecter_satellite_mission'
        );

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            RAISE_APPLICATION_ERROR(
                -20048,
                'Affectation déjà existante pour ce satellite et cette
mission.'
            );
        WHEN OTHERS THEN
            RAISE;
    END affecter_satellite_mission;


    ----------------------------------------------------------------
    -- mettre_en_revision
    -- Hypothèse retenue : un satellite mis en révision passe en "En
veille".
    -- Le trigger T5 historise automatiquement le changement.
    ----------------------------------------------------------------
    PROCEDURE mettre_en_revision(
        p_id_satellite IN SATELLITE.id_satellite%TYPE
    )
    IS
        v_statut SATELLITE.statut%TYPE;
    BEGIN
        SELECT statut
        INTO v_statut
        FROM SATELLITE
        WHERE id_satellite = p_id_satellite
        FOR UPDATE;
```

```
        IF v_statut = 'Désorbité' THEN
            RAISE_APPLICATION_ERROR(
                -20049,
                'Un satellite désorbité ne peut pas être mis en
révision.'
            );
        END IF;

        UPDATE SATELLITE
        SET statut = 'En veille'
        WHERE id_satellite = p_id_satellite;

        IF SQL%ROWCOUNT = 0 THEN
            RAISE_APPLICATION_ERROR(
                -20050,
                'Aucune mise à jour du satellite effectuée.'
            );
        END IF;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(
                -20051,
                'Satellite introuvable.'
            );
        WHEN OTHERS THEN
            RAISE;
    END mettre_en_revision;


    ----------------------------------------------------------------
    -- calculer_volume_theorique
    -- Volume théorique = débit maximal station * durée
    ----------------------------------------------------------------
    FUNCTION calculer_volume_theorique(
        p_id_fenetre IN FENETRE_COM.id_fenetre%TYPE
    ) RETURN NUMBER
    IS
        v_duree   FENETRE_COM.duree%TYPE;
        v_debit   STATION_SOL.debit_max%TYPE;
        v_volume  NUMBER;
    BEGIN
        SELECT f.duree, s.debit_max
        INTO v_duree, v_debit
```

```
        FROM FENETRE_COM f
        JOIN STATION_SOL s
          ON s.code_station = f.code_station
        WHERE f.id_fenetre = p_id_fenetre;

        v_volume := v_duree * v_debit;
        RETURN v_volume;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(
                -20052,
                'Fenêtre introuvable pour le calcul du volume
théorique.'
            );
    END calculer_volume_theorique;


    ----------------------------------------------------------------
    -- statut_constellation
    -- Retourne un résumé textuel : x/y satellites opérationnels, y
missions actives
    ----------------------------------------------------------------
    FUNCTION statut_constellation
    RETURN VARCHAR2
    IS
        v_nb_sat_op      NUMBER;
        v_nb_sat_total   NUMBER;
        v_nb_miss_act    NUMBER;
    BEGIN
        SELECT COUNT(*)
        INTO v_nb_sat_total
        FROM SATELLITE;

        SELECT COUNT(*)
        INTO v_nb_sat_op
        FROM SATELLITE
        WHERE statut = 'Opérationnel';

        SELECT COUNT(*)
        INTO v_nb_miss_act
        FROM MISSION
        WHERE statut_mission = 'Active';
```

```
        RETURN v_nb_sat_op || '/' || v_nb_sat_total ||
               ' satellites opérationnels, ' ||
               v_nb_miss_act || ' missions actives';

    EXCEPTION
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(
                -20053,
                'Erreur lors du calcul du statut de la constellation : '
|| SQLERRM
            );
    END statut_constellation;


    ----------------------------------------------------------------
    -- stats_satellite
    -- Retourne des statistiques agrégées sur un satellite :
    -- nb_fenetres, volume_total, duree_moy_secondes
    ----------------------------------------------------------------
    FUNCTION stats_satellite(
        p_id_satellite IN SATELLITE.id_satellite%TYPE
    ) RETURN t_stats_satellite
    IS
        v_stats t_stats_satellite;
        v_dummy SATELLITE.id_satellite%TYPE;
    BEGIN
        SELECT id_satellite
        INTO v_dummy
        FROM SATELLITE
        WHERE id_satellite = p_id_satellite;

        SELECT COUNT(*),
               NVL(SUM(volume_donnees), 0),
               NVL(AVG(duree), 0)
        INTO v_stats.nb_fenetres,
             v_stats.volume_total,
             v_stats.duree_moy_secondes
        FROM FENETRE_COM
        WHERE id_satellite = p_id_satellite;

        RETURN v_stats;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
```

```
            RAISE_APPLICATION_ERROR(
                -20054,
                'Satellite introuvable pour le calcul des statistiques.'
            );
    END stats_satellite;

END pkg_nanoOrbit;
/
SHOW ERRORS;
```

Vérification de compilation

```
Package body created.
```

Analyse

La procédure planifier_fenetre insère une nouvelle fenêtre tout en laissant aux triggers de phase 2 le soin d'appliquer les contrôles de cohérence métier, notamment sur le statut du satellite, le statut de la station et les chevauchements temporels.

La procédure cloturer_fenetre traduit la transition logique entre une fenêtre Planifiée et une fenêtre Réalisée, conformément au modèle de données où volume_donnees n'est renseigné qu'après réalisation.

La procédure affecter_satellite_mission permet d'enregistrer une participation satellite–mission dans la table PARTICIPATION, en cohérence avec le rôle métier porté par cette association.

La procédure mettre_en_revision repose sur une mise à jour du statut satellite et laisse au trigger d'historisation la responsabilité d'enregistrer automatiquement l'événement dans HISTORIQUE_STATUT.

La fonction calculer_volume_theorique retourne un calcul agrégé fondé sur la durée de la fenêtre et le débit maximal de la station associée, selon la logique débit × durée.

La fonction statut_constellation synthétise l'état global du système en restituant un résumé textuel du nombre de satellites opérationnels et de missions actives, conformément à l'esprit du sujet.

Enfin, la fonction stats_satellite retourne un enregistrement structuré du type public t_stats_satellite, permettant d'exposer proprement les statistiques d'un satellite donné.

Conclusion

Le BODY du package pkg_nanoOrbit permet de centraliser les principales opérations métier du système NanoOrbit dans une interface PL/SQL cohérente et réutilisable. Il constitue le support direct du livrable L3-D, consacré au scénario de validation du package.

4) Script de validation

Le livrable L3-D consiste à produire un scénario de validation complet du package pkg_nanoOrbit. L'objectif est de démontrer que les sous-programmes déclarés dans la SPEC et implémentés dans le BODY fonctionnent correctement dans un enchaînement réaliste d'opérations métier. Le sujet demande explicitement un bloc de test enchaîné avec des sorties DBMS_OUTPUT commentées.

Le scénario de validation a été construit autour des opérations suivantes :

- planification d'une nouvelle fenêtre de communication pour SAT-001 vers GS-KIR-01
- calcul du volume théorique de cette fenêtre
- clôture de la fenêtre avec un volume réel
- affectation de SAT-004 à la mission MSN-ARC-2023
- mise en révision d'un satellite
- récupération des statistiques de SAT-001
- affichage du résumé global de la constellation

Ce scénario réutilise les identifiants du jeu de données officiel NanoOrbit. Le choix de SAT-001 et GS-KIR-01 est cohérent avec le référentiel, car SAT-001 est opérationnel et GS-KIR-01 est active. À l'inverse, des cas comme SAT-005 ou GS-SGP-01 ont été évités, car ils sont explicitement bloqués par les règles métier et les triggers de phase 2. SAT-005 est désorbité et GS-SGP-01 est en maintenance.

Code

```
SET SERVEROUTPUT ON;

-- ==========================================================
-- L3-D : Script de validation du package pkg_nanoOrbit
-- Scénario enchaîné de test avec affichages DBMS_OUTPUT
-- ==========================================================

DECLARE
    v_id_fenetre       FENETRE_COM.id_fenetre%TYPE;
    v_volume_theorique NUMBER;
    v_stats            pkg_nanoOrbit.t_stats_satellite;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== Début du scénario de validation
pkg_nanoOrbit ===');
```

```oracle_plsql
    ----------------------------------------------------------------
    -- 1) Planifier une nouvelle fenêtre de communication
    -- Créneau déplacé pour éviter tout chevauchement existant
    ----------------------------------------------------------------
    pkg_nanoOrbit.planifier_fenetre(
        p_id_satellite   => 'SAT-001',
        p_code_station   => 'GS-KIR-01',
        p_datetime_debut => TIMESTAMP '2024-01-25 16:30:00',
        p_duree          => 300,
        p_id_fenetre     => v_id_fenetre
    );

    DBMS_OUTPUT.PUT_LINE('Fenêtre planifiée avec succès. ID = ' ||
v_id_fenetre);

    ----------------------------------------------------------------
    -- 2) Calculer le volume théorique de la fenêtre
    ----------------------------------------------------------------
    v_volume_theorique :=
pkg_nanoOrbit.calculer_volume_theorique(v_id_fenetre);

    DBMS_OUTPUT.PUT_LINE(
        'Volume théorique de la fenêtre ' || v_id_fenetre || ' = ' ||
v_volume_theorique
    );

    ----------------------------------------------------------------
    -- 3) Clôturer la fenêtre en renseignant un volume réel
    ----------------------------------------------------------------
    pkg_nanoOrbit.cloturer_fenetre(
        p_id_fenetre     => v_id_fenetre,
        p_volume_donnees => 1180
    );

    DBMS_OUTPUT.PUT_LINE(
        'Fenêtre ' || v_id_fenetre || ' clôturée avec volume réel =
1180'
    );

    ----------------------------------------------------------------
    -- 4) Affecter SAT-004 à la mission MSN-ARC-2023
    ----------------------------------------------------------------
    pkg_nanoOrbit.affecter_satellite_mission(
        p_id_satellite => 'SAT-004',
```

```
        p_id_mission   => 'MSN-ARC-2023',
        p_role         => 'Satellite de relais'
    );

    DBMS_OUTPUT.PUT_LINE(
        'SAT-004 affecté à MSN-ARC-2023 avec le rôle "Satellite de
relais".'
    );

    ----------------------------------------------------------------
    -- 5) Mettre SAT-003 en révision
    ----------------------------------------------------------------
    pkg_nanoOrbit.mettre_en_revision('SAT-003');

    DBMS_OUTPUT.PUT_LINE('SAT-003 a été mis en révision.');

    ----------------------------------------------------------------
    -- 6) Récupérer les statistiques de SAT-001
    ----------------------------------------------------------------
    v_stats := pkg_nanoOrbit.stats_satellite('SAT-001');

    DBMS_OUTPUT.PUT_LINE('Statistiques de SAT-001 :');
    DBMS_OUTPUT.PUT_LINE('- Nombre de fenêtres       : ' ||
v_stats.nb_fenetres);
    DBMS_OUTPUT.PUT_LINE('- Volume total             : ' ||
v_stats.volume_total);
    DBMS_OUTPUT.PUT_LINE('- Durée moyenne (secondes) : ' ||
v_stats.duree_moy_secondes);

    ----------------------------------------------------------------
    -- 7) Afficher le statut global de la constellation
    ----------------------------------------------------------------
    DBMS_OUTPUT.PUT_LINE(
        'Résumé constellation : ' || pkg_nanoOrbit.statut_constellation
    );

    DBMS_OUTPUT.PUT_LINE('=== Fin du scénario de validation
pkg_nanoOrbit ===');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur pendant le scénario : ' ||
SQLERRM);
        RAISE;
```

```sql
END;
/
SHOW ERRORS;

-- ============================================================
-- Vérifications complémentaires éventuelles
-- ============================================================

-- Vérifier la fenêtre créée
SELECT id_fenetre, datetime_debut, duree, statut, volume_donnees,
id_satellite, code_station
FROM FENETRE_COM
WHERE id_fenetre = [ID_FENETRE_GENERE];

-- Vérifier l'affectation de mission
SELECT id_satellite, id_mission, role_satellite
FROM PARTICIPATION
WHERE id_satellite = 'SAT-004'
  AND id_mission = 'MSN-ARC-2023';

-- Vérifier l'historique après mise en révision
SELECT id_historique, statut, timestamp_, id_satellite
FROM HISTORIQUE_STATUT
WHERE id_satellite = 'SAT-003'
ORDER BY timestamp_ DESC;
```

Résultat obtenu

```
=== Début du scénario de validation pkg_nanoOrbit ===
Fenêtre planifiée avec succès. ID = 9
Volume théorique de la fenêtre 9 = 120000
Fenêtre 9 clôturée avec volume réel = 1180
SAT-004 affecté à MSN-ARC-2023 avec le rôle "Satellite de relais".
SAT-003 a été mis en révision.
Statistiques de SAT-001 :
- Nombre de fenêtres       : 3
- Volume total             : 2430
- Durée moyenne (secondes) : 366.6666666666666666666666666666666667
Résumé constellation : 2/5 satellites opérationnels, 2 missions actives
=== Fin du scénario de validation pkg_nanoOrbit ===
```

Analyse

La validation du package montre que les différents sous-programmes coopèrent correctement avec les mécanismes déjà présents dans le schéma NanoOrbit, notamment les triggers de phase 2. La procédure planifier_fenetre s'appuie sur les contrôles métier de FENETRE_COM, la procédure mettre_en_revision déclenche l'historisation automatique du changement de statut, et les fonctions stats_satellite et statut_constellation restituent des résultats synthétiques exploitables.

Le scénario valide donc le package sur l'ensemble de son périmètre fonctionnel.

Conclusion

Le livrable L3-D confirme que le package pkg_nanoOrbit constitue une couche métier PL/SQL cohérente et réutilisable. Il permet de centraliser des opérations complexes du système NanoOrbit dans une interface unique, tout en respectant les règles de cohérence déjà implantées dans la base Oracle.